

Credit Card Approval Prediction: ML-Based Credit Risk Classifier

Project Description:

Credit Card Approval Prediction is a machine learning application that automates the assessment of whether a credit card applicant is likely to be a 'Good Customer' or a 'Bad Customer' based on their demographic and financial profile. The system merges two datasets — applicant details (income, employment, family status, housing, etc.) and historical monthly credit/payment records — to build a labeled dataset, then trains and compares three classification models (Logistic Regression, Decision Tree, and Random Forest) to identify the best-performing approach for this credit-risk classification task.

The trained Random Forest model is serialized and served through a Flask web application, which presents applicants (or an evaluator/loan officer) with a simple form to enter details such as gender, income, education, family status, housing type, age, and employment experience. On submission, the backend converts these inputs into the same feature format used during training, feeds them through the model, and instantly returns a Good Customer / Bad Customer prediction — giving lenders a fast, data-driven first pass on credit card applications.

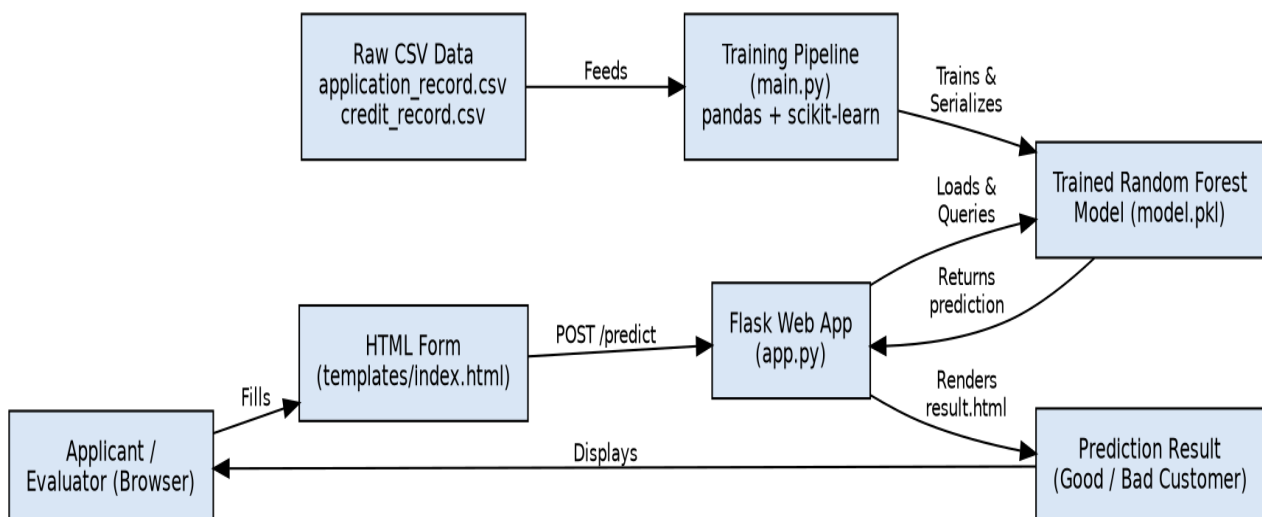
Scenarios

Scenario 1: A loan officer reviewing a new applicant — a salaried employee with a stable income, higher education, and owned property — enters these details into the web form. The model classifies the applicant as a “Good Customer”, supporting a fast-tracked approval recommendation.

Scenario 2: An applicant with a lower declared income, a shorter employment history, and no owned real estate submits their details. The Random Forest model weighs these risk factors and flags the applicant as a “Bad Customer”, prompting manual review before any credit line is extended.

Scenario 3: A bank runs a batch of historical applications through the same pipeline used in **main.py** to validate model accuracy against known outcomes, using the Accuracy, F1-score, and confusion matrix output to confirm the Random Forest model still outperforms Logistic Regression and Decision Tree before redeploying an updated **model.pkl** to production.

Technical Architecture:



Description: The application follows a simple two-stage architecture: an offline **training pipeline** and an online **inference/serving layer**. The training pipeline (**main.py**) loads the raw CSVs with pandas, engineers the target label from credit history, label-encodes categorical fields, and trains/evaluates three scikit-learn classifiers before persisting the best model (Random Forest) as **model.pkl**. The serving layer is a lightweight Flask app (**app.py**) that renders an HTML form (**templates/index.html**), accepts a POST request on **/predict**, reconstructs the feature vector in the exact order used during training, loads **model.pkl**, and renders the prediction on **templates/result.html**.

Note: This project does not use a database. All data is sourced from two static CSV files (**application_record.csv** and **credit_record.csv**) that are loaded directly by the training script, so an ER diagram is not applicable here.

Pre-requisites:

- **Python Programming Proficiency:** Python 3.8+ fundamentals, including working with functions and scripts
- **pandas & NumPy:** data loading, merging, cleaning, and feature engineering on tabular data
- **scikit-learn:** LabelEncoder, train_test_split, LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, and evaluation metrics (accuracy, F1-score, confusion matrix)
- **Flask Framework Knowledge:** routing, request handling, and render_template
- **HTML & CSS Skills:** for building and styling the applicant input form and results page
- **Version Control with Git:** cloning and managing the project repository
- **Basic Understanding of Credit Risk Concepts:** helpful for interpreting the Good/Bad Customer classification and evaluation metrics

Project Workflow:

Milestone 1: Data Understanding and Architecture

- **Activity 1.1:** Load and inspect the two source datasets (application_record.csv and credit_record.csv) with pandas; review shapes, data types, and sample rows.
- **Activity 1.2:** Research and select suitable classification algorithms for tabular credit-risk data — Logistic Regression (baseline, interpretable), Decision Tree (non-linear, interpretable), and Random Forest (ensemble, typically stronger accuracy).
- **Activity 1.3:** Define the application architecture — an offline training pipeline (main.py) that produces a serialized model, and an online Flask serving layer (app.py) that consumes it.
- **Activity 1.4:** Set up the development environment and install the required libraries: pandas, numpy, scikit-learn, and flask.

Milestone 2: Data Preprocessing and Feature Engineering

Activity 2.1: Clean and Prepare the Application Data

- Remove duplicate applicant rows with app.drop_duplicates().
- Check for missing values across all columns using app.isnull().sum().
- Fill missing OCCUPATION_TYPE entries with the placeholder value "Unknown" rather than dropping rows.

```
app.drop_duplicates(inplace=True)
app["OCCUPATION_TYPE"] = app["OCCUPATION_TYPE"].fillna("Unknown")
```

Activity 2.2: Engineer the Target Label from Credit History

- Each customer has multiple monthly credit_record rows with a STATUS code. Map STATUS values 1–5 (overdue by varying degrees) to a "bad" label (1) and C / X / 0 to a "good" label (0).
- Collapse each customer's monthly records into a single row per ID using the worst (maximum) observed status with groupby("ID")["TARGET"].max().

```
credit["TARGET"] = credit["STATUS"].apply(
    lambda x: 1 if x in ["1", "2", "3", "4", "5"] else 0
)
credit = credit.groupby("ID")["TARGET"].max().reset_index()
```

Activity 2.3: Merge Datasets and Encode Categorical Features

- Inner-join the application data with the aggregated credit target on the ID column.
- Label-encode all categorical columns (CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE, OCCUPATION_TYPE) so the models can consume them numerically.

```
data = pd.merge(app, credit, on="ID", how="inner")

categorical_columns = [
    'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY',
    'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE',
    'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE', 'OCCUPATION_TYPE'
]
encoder = LabelEncoder()
for column in categorical_columns:
    data[column] = encoder.fit_transform(data[column])
```

Milestone 3: Model Training and Evaluation (main.py)

Activity 3.1: Split the Data

Separate features (X) from the target (y) and split into an 80/20 train-test split, stratified on the target to preserve the good/bad customer ratio in both sets.

```
X = data.drop(columns=["ID", "TARGET"])
y = data["TARGET"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Activity 3.2: Train and Evaluate Three Candidate Models

- **Logistic Regression:** a linear baseline model, trained with max_iter=1000.
- **Decision Tree:** a single-tree, non-linear classifier for comparison.
- **Random Forest:** an ensemble of 100 trees (n_estimators=100), used as the final production model.
- Each model is scored on Accuracy and F1-score, with a full confusion matrix and classification report printed for inspection.

```
rf_model = RandomForestClassifier(
    n_estimators=100, random_state=42, n_jobs=-1
)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred)
```

Activity 3.3: Compare Models and Persist the Best One

Accuracy and F1-score for all three models are collected into a comparison table; the Random Forest model is selected as the best performer and serialized to model.pkl using pickle for later use by the Flask app.

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [lr_accuracy, dt_accuracy, rf_accuracy],
    "F1 Score": [lr_f1, dt_f1, rf_f1]
})

best_model = rf_model
pickle.dump(best_model, open("model.pkl", "wb"))
```

Milestone 4: Flask Backend Development (app.py)

Activity 4.1: Define Application Routes

- `/` — renders the home page (index.html) containing the applicant input form.
- `/predict` (POST) — accepts the submitted form data, builds the feature vector, runs the prediction, and renders the result page.

```
@app.route("/")
def home():
    return render_template("index.html")
```

Activity 4.2: Process and Transform Form Inputs

The form collects the applicant's name (display only) plus 17 numeric/encoded fields matching the training feature set. Age and years of experience are converted into the DAYS_BIRTH and DAYS_EMPLOYED convention used by the original dataset (negative day counts).

```
age = int(request.form["age"])
experience = int(request.form["experience"])

days_birth = -(age * 365)
```

```
days_employed = -(experience * 365)

features = np.array([[
    gender, car, house, children, income, income_type,
    education, family_status, housing, days_birth,
    days_employed, mobile, work_phone, phone, email,
    occupation, family_members
]])
```

Activity 4.3: Run Inference and Render the Result

The pickled Random Forest model is loaded once at startup. On each request, it predicts a class (0 = Good Customer, 1 = Bad Customer) for the constructed feature array, and the outcome is passed to result.html along with the applicant's name.

```
model = pickle.load(open("model.pkl", "rb"))

prediction = model.predict(features)[0]
result = "Good Customer" if prediction == 0 else "Bad Customer"

return render_template("result.html", prediction=result, applicant=name)
```

Milestone 5: Frontend Development

- **Input Form (templates/index.html):** a single-page HTML form collecting applicant name, gender, car/property ownership, number of children, income, income type, education, family status, housing type, age, years of employment experience, contact flags (mobile/work phone/phone/email), occupation, and family member count.
- **Result Page (templates/result.html):** displays the applicant's name alongside the predicted outcome ("Good Customer" or "Bad Customer") returned by the /predict route.
- **Styling (static/):** CSS assets used to style the form and result page for a clean, readable layout.

Milestone 6: Deployment

Activity 6.1: Local Deployment and Testing

- Install dependencies: `pip install pandas numpy scikit-learn flask`.
- Generate/refresh the model artifact by running `python main.py` (produces `model.pkl` from the CSV datasets).
- Start the Flask development server with `python app.py` and verify predictions at `http://127.0.0.1:5000` by submitting a range of applicant profiles.

Activity 6.2: Public Deployment (Render)

The project is intended to be deployed publicly on **Render**. This requires adding a `requirements.txt` (pandas, numpy, scikit-learn, flask, gunicorn), disabling Flask's debug mode for production, and configuring Render's web service to run the app with a production WSGI server (e.g. `gunicorn app:app`). Deployment is currently planned but not yet live.

Exploring the Website's Web Pages:

Home / Applicant Form Page:

Description: The landing page renders a single form where a loan officer or applicant enters all details needed for a prediction — personal/demographic information (gender, age, family status, education, number of children/family members), financial information (income, income type), asset ownership (car, property/house), housing type, employment experience, contact flags, and occupation. Submitting the form sends a POST request to /predict.

Result Page:

Description: After submission, this page displays the applicant's name and the model's prediction — either "Good Customer" or "Bad Customer" — giving an immediate, readable outcome for the person who submitted the form.

Conclusion:

Credit Card Approval Prediction demonstrates an end-to-end applied machine learning workflow: merging and engineering a labeled dataset from raw applicant and credit-history records, comparing multiple classification algorithms, and serving the best-performing model (Random Forest) through a lightweight Flask web application. The project shows how a data pipeline built with pandas and scikit-learn can be paired with a simple web interface to turn a trained model into an interactive credit-risk screening tool. Looking ahead, the project could be strengthened by adding a proper preprocessing pipeline (saving encoders alongside the model), addressing class imbalance more explicitly with metrics like ROC-AUC, adding a `requirements.txt` and README for reproducibility, and completing public deployment on Render with debug mode disabled for production use.